Graphics Programming

“Hello, World”

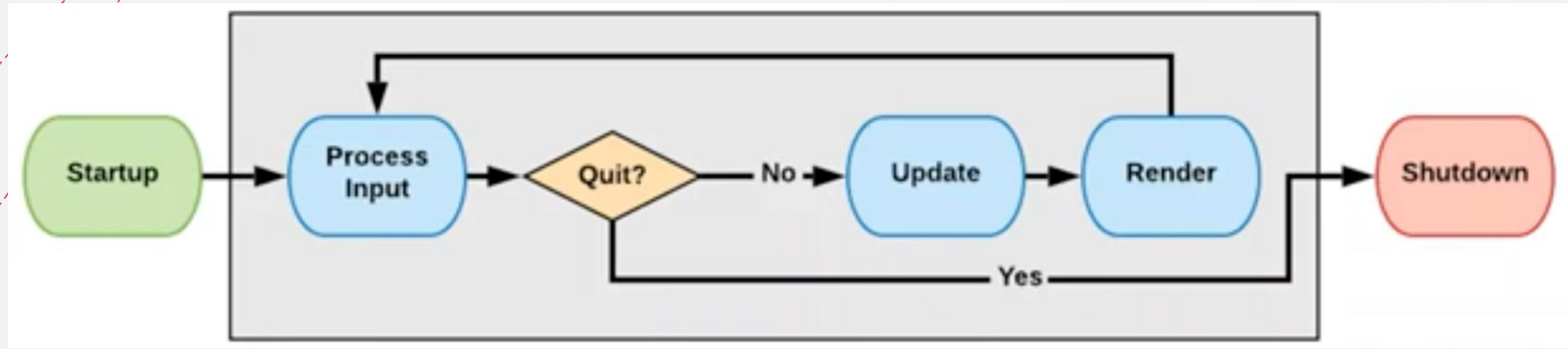
GB. Tony Liang

Recap: The Graphics Pipeline

- An abstract model, describes a sequence of steps to render a three-dimensional scene
- Allows a computational task to be split into subtasks
  - Each of which can be worked on in parallel

Recap: The Graphics Pipeline

- **Application**
  - Initializing the window where graphics will be displayed
  - Sending data to the GPU
- **Geometry Processing**
  - Determining the position of each vertex of the geometric shapes to be rendered
- **Rasterization**
  - Determining which pixels correspond to the geometric shapes to be rendered
- **Pixel Processing**
  - Determining the color of each pixel in the rendered image

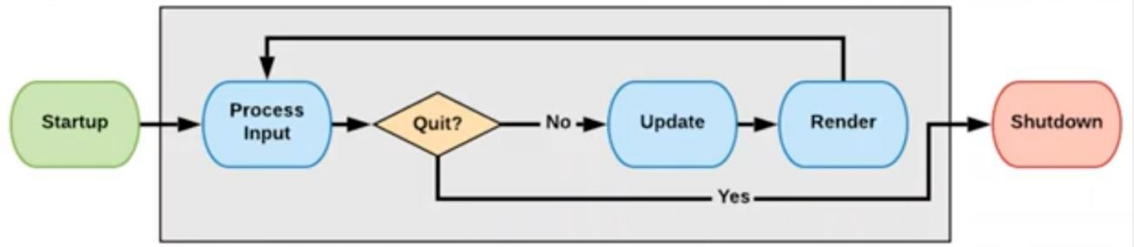


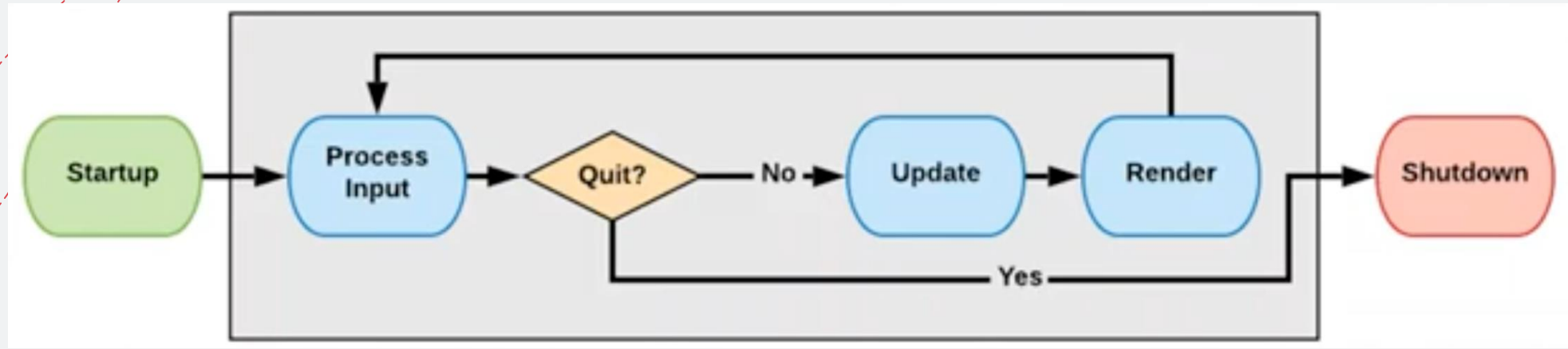
Application Life Cycle

- **Startup**
  - Object created, values initialized, external files loaded

Application Life Cycle

- **Main loop**
  - Repeats continuously (typically 60 FPS for computer)
  - Consists of three sub-stages:
    - Process Input
    - Update
    - Render





Application Life Cycle

- **Shutdown**
  - Stop checking for user input, closing application windows

OpenGL Shading Language

OpenGL Shading Language (GLSL)

- Basic data type
  - *bool,*
  - *int,*
  - *float,*
  - *vecN,*
  - *matN*

OpenGL Shading Language (GLSL)

- *vec4 v*
  - Access elements as array
    - v[0], v[1], v[2], v[3]
  - Or dot notation
    - *v.x, v.y, v.z, v.w* OR *v.r, v.g, v.b, v.a*

OpenGL Shading Language (GLSL)

+ Shaders must contain main functions:

```
void main(){
        // code
}
```

Passing data to shaders

- **Data passed in and out shaders by variables that are declared with *type qualifiers***
  - Keywords that modify the properties or a variable
  - *in*: variable values supplied by the previous pipeline stage
  - *out:* variable value will be passed to the next pipeline stage
- **Vertex shader**
  - *in*: values supplied from a buffer
  - *out*: values passed to fragment shader
- **Fragment shader**
  - *in*: values supplied from vertex shader (interpolated)
  - *out*: values stored in a buffer (color, depth, stencil)

The Simplest Shader

- **Vertex shader must set value**: *vec4 gl_Position*
- **Fragment shader must set value:** *vec4 fragColor*
  - The pre-fix of “gl_” are reserved for built-in variables
  - Older OpenGL (e.g., 1.20/1.30) had a build-in fragment shader variable (*gl_FragColor*).
  - OpenGL 3.30 removed the build-in fragment shader variable since it allows multiple fragment shader outputs

The Simplest Shader

- **Shader code to render a yellow point in a center of screen:**

```
void main()    // vertex shader
{
        gl_Position = vec4(0.0, 0.0, 0.0, 1.0);
}

void main()    // fragment shader
{
        fragColor = vec4(1.0, 1.0, 0.0, 1.0);
}
```

Compiling GPU Programs

Shaders (1/2)

- glCreateShader *(shaderType)*
  - Create an empty shader object, used to store the source code of a shader, and returns a value by which it can be referenced.
    - It does not return a shader object, but return the reference.
  - The type of shader (such as a vertex shader or a fragment shader) is specified with the *shaderType* parameter.
    - Value will be an OpenGL constant:
      - GL_VERTEX_SHADER or GL_FRAGMENT_SHADER

Shaders (2/2)

- glShaderSource (*shaderRef, shaderCode*)
  - Stores the source code in the string parameter *shaderCode* in the shader object
  - Referenced by the parameter *shaderRef*

- glCompileShader (*shaderRef*)
  - Compiles the source code stored in the shader object
  - Referenced by the parameter *shaderRef*

Shader Error Checking (1/2)

- glGetShaderiv(*shaderRef, shaderInfo*)
  - Returns information from the shader referenced by the parameter *shaderRef*
  - The type of information retrieved is specified with the *shaderInfo* parameter, whose value will be an OpenGL constant, such as GL_SHADER_TYPE (to determine the type of shader) or GL_COMPILE_STATUS (to determine if compilation was successful).

Shader Error Checking (2/2)

- glGetShaderInfoLog(*shaderRef*)
  - Returns information about the compilation process (such as errors and warnings) from the shader referenced by the parameter *shaderRef*

- glDeleteShader (*shaderRef*)
  - Frees the memory used by the shader referenced by the parameter *shaderRef*
  - Makes the reference available for future shaders

GPU Programs

- **glCreateProgram()**
  - Creates an empty program object, to which shader objects can be attached
  - Return a value by which it can be referenced
- **glAttachShader(*programRef*, *shaderRef*)**
  - Attaches a shader object (i.e., a compiled shader) specified by the parameter *shaderRef* to the program object specified by the parameter *programRef*
- **glLinkProgram(*programRef*)**
  - Links the vertex and fragment shaders to the program object specified by the parameter *programRef*

GPU Programs Error Checking

- **glGetProgramiv(*programRef*, *programInfo*)**
  - Returns information from program referenced by *programRef*
  - The type of retrieved is specified with *programInfo*
    - E.g., GL_LINK_STATUS to determine if linking was successful
- **glGetProgramLog(*programRef*)**
  - Returns information about the linking process (such as errors and warnings) from the program referenced by *programRef*
- **glDeleteProgram (*programRef*)**
  - Frees memory used by the program referenced by *programRef*
  - Make the reference available for future programs

Rendering in Application

OpenGL functions

- glUseProgram(*programRef*)
  - Specifies use of program referenced by *programRef* when rendering
- glDrawArray(*drawMode, firstIndex, indexCount*)
  - Renders geometric primitives
    - Points, lines, or triangles, specified by *drawMode*
  - Uses data from array stored in vertex buffers beginning at *firstIndex*
  - Total count of the element is *indexCount*
- glPointSize(*size*)
  - Specifies points should be rendered with diameter (in pixels) size